Algoritmalar ve Veri Yapıları Bootcamp Ödev 4.

1. Dinamik programlama ile para üstü problemini çözen algoritma, istenen para üstünü tamamlamak için kullanılabilecek minimum madeni para adedini döndürüyor, ancak madeni paraların türü (yani, algoritmaya girdi olarak verilen D dizisinin hangi elemanlarının çözümüm parçası olduğu bilgisi) hakkında hiçbir şey söylemiyor. Derste bu paraların hangileri olabileceğini gözlem yoluyla tespit etmiştik. Bu ödevde, dinamik programlama ile para üstü problemini çözen algoritmada gerekli değişiklikleri yaparak, algoritmayı bulduğu optimum çözümde yer alan madeni paraların adedini değil de değerlerini bir dizi şeklinde döndürecek hale getiriniz. Örneğin dersteki örnek üzerinden çalıştırdığınızda algoritmanız [3, 3] sonucunu dönmeli. Algoritmanızın zaman ve uzay karmaşıklıklarını ayrı ayrı belirtin.

2. Elimizde girdi olarak n adet pozitif tamsayı a_1, a_2, …., a_n ve başka bir pozitif tamsayı olan B değeri olduğunu düşünelim. Bu sayıların toplamları B'ye eşit olan bir **alt kümesinin** olup olmadığı sorusunu yanıtlayan bir Dinamik Programlama algoritması tasarlayın ve pseudo-kod olarak uygulayın. Eğer bu sayıların herhangi bir alt kümesindeki sayıların (alt küme elemanlarının) toplamları B'ye eşit olabiliyorsa algoritmanız True döndürmeli, hiç bir alt küme elemanlarının toplamı B'ye eşit olamıyorsa False döndürmelidir. Örneğin:

Örnek 1: $n=3, a_1=5, a_2=7, a_3=4, B=9.$	Cevap: True, zira 5+4=9.
Örnek 2: $n=3, a_1=5, a_2=7, a_3=4, B=20.$	Cevap: False, zira bütün elemanları dahi toplasak çıkan değer 16 < 20.

Verilen sayıların tamamının pozitif olduğu bilgisi algoritmanızı nasıl yönlendiriyor üzerinde düşünmelisiniz. Algoritmanızın kullandığı tabloyu açıkça belirtin ve çözüme ulaşana kadar algoritmanızın her adımını gösteriniz. Son olarak, algoritmanızın en kötü durum zaman karmaşıklığını da analiz ediniz.

İpucu: Dinamik programlama ile problemi çözmek için, daha küçük problemlerin çözümlerini birleştirerek çözebileceğimiz bir yineleme denklemi bulmamız gerekiyor. Yukarıdaki problemin daha küçük boyutlu problemleri için şu şekilde düşünebilirsiniz:

$1 <= i <= n$ oluşan ve $a_1, a_2, a_3, ...a_n$ değerlerine sahip tam sayılar
$1 <= j <= B,$ kontrol etmek istediğimiz toplam ağırlıklar olduğunda

i tam sayıdan oluşan bir (alt)küme ve toplam j için problemin en uygun çözümünü (true veya false/olarak) F(i, j) ile gösterelim. İlk i tam sayının tüm olası alt kümelerini iki gruba ayırabiliriz:

- *i. (son) tamsayıyı içeren alt kümeler ve*
- *i. (son) tamsayıyı içermeyen alt kümeler.*

Bundan sonrası sizde 🙂